

CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

Monitoring and Analyzing Cyber Security Attacks in
Microservice Applications using AWS

A thesis submitted in partial fulfillment of the
requirements for the degree of Master of Science in
Computer Science

By
Jahanzaib Malik

May 2023

The thesis of Jahanzaib Malik is approved:

Dr. Robert McIlhenny

Date

Dr. Kyle Dewey

Date

Dr. Wonjun Lee, Chair

Date

Acknowledgement

I am deeply grateful to Dr. Wonjun Lee, my committee chair, for his unwavering support, guidance, and mentorship throughout this journey. His expertise and dedication have been invaluable to the completion of this thesis.

My heartfelt appreciation goes to my committee members, Dr. Robert McIlhenny and Dr. Kyle Dewey, for their valuable insights, feedback, and encouragement. Their expertise has significantly contributed to the development of this work.

To my parents, thank you for your unwavering belief in my abilities, support, and love. Your sacrifices and faith in me have been my anchor during this challenging journey.

My younger brother, Aurangzaib Malik, has been a constant source of strength, motivation, and humor. I am incredibly lucky to have you as my brother and best friend.

Lastly, I acknowledge my friends, colleagues, and the academic community for their camaraderie, shared wisdom, and encouragement. I am grateful for the opportunity to learn and grow alongside such an incredible group of individuals.

I dedicate this thesis to all who have supported and inspired me throughout this journey. Your belief in me has made this achievement possible, and I am forever grateful.

Table of Contents

Signature Page.....	ii
Acknowledgement.....	iii
List of Figures.....	v
Abstract.....	vii
Chapter 1 - Introduction.....	1
1.1 Background and motivation.....	1
1.2 Problem statement.....	2
1.3 Objectives and scope.....	3
1.4 Thesis organization.....	4
Chapter 2 - Literature Review.....	6
2.1 Microservice architecture.....	6
2.2 Cybersecuriy attacks on microservices.....	7
2.3 Monitoring and Logging in Microservices.....	8
2.4 AWS and its relevant services.....	9
2.5 Deep learning models for attack detection and classification.....	10
2.6 Related work.....	12
Chapter 3 – Methodology.....	14
3.1 Deploying the "Sock-Shop" microservice application to AWS EKS.....	14
3.1.1 AWS EKS configuration and setup.....	14
3.1.2 Deployment of "sock-shop" to the cluster.....	17
3.2 Modifying the "sock-shop" interface for cybersecurity attacks.....	18
3.3 Attack generation and simulation.....	22
3.4 Collecting logs using AWS CloudWatch and Preprocessing.....	26
3.5 Deep learning model selection and implementation.....	31
Chapter 4 – Results.....	37
Chapter 5 - Conclusion and Future Work.....	42
Chapter 6 - References.....	43

List of Figures

Figure 1. eksadmin Policy.....	15
Figure 2. IAM Role Validation.....	15
Figure 3. Create Cluster.....	16
Figure 4. ConfigMap MapUsers.....	17
Figure 5. KubeConfig.....	17
Figure 6. Clone Sock-Shop.....	18
Figure 7. Create Namespace.....	18
Figure 8. Apply Kubernetes Manifest.....	18
Figure 9. Clone Front-End.....	19
Figure 10. Build Front-End.....	19
Figure 11. Docker Push Front-End.....	19
Figure 12. Clone Sock-Shop.....	19
Figure 13. Docker Compose File.....	20
Figure 14. Kubernetes Manifest Download.....	21
Figure 15. Kubernetes Manifest.....	21
Figure 16. Python Encoding Script.....	23
Figure 17. BurpSuite.....	24
Figure 18. JMeter Setup.....	25
Figure 19. JMeter Load Test.....	26
Figure 20. FluentBit Setup.....	27
Figure 21. Log Groups.....	28
Figure 22. Container Insights.....	28
Figure 23. Container Insight Metrics.....	29
Figure 24. Merge Dataframe.....	30
Figure 25. Scale and Normalize.....	30
Figure 26. Preprocessing.....	30
Figure 27. Dataset(s) Details.....	31
Figure 28. LSTM Preprocessing.....	32
Figure 29. Build LSTM Model.....	32
Figure 30. Analysing LSTM Model.....	32
Figure 31. Autoencoder Preprocessing.....	34
Figure 32. Building Autoencoder.....	35
Figure 33. Threshold Calculation.....	35

Figure 34. Analysing Autoencoder.....	35
Figure 35. HeatMap Generation.....	36
Figure 36. SMOTE.....	36
Figure 37. LSTM Results.....	38
Figure 38. Autoencoder Results.....	39
Figure 39. Autoencoder with SMOTE Results.....	40

Abstract

Monitoring and Analyzing Cyber Security Attacks in Microservice Applications using AWS

By

Jahanzaib Malik

Master of Science in Computer Science

As the major corporations are shifting from monolithic architectures to more scalable and flexible microservice architectures, this has introduced new challenges in terms of cyber security. Microservices consist of numerous small interconnected services, thus the term Micro-Services, which are more vulnerable to cyber attacks due to their distributed nature. This paper attempts to monitor and analyze cybersecurity attacks on these microservice applications deployed on cloud platforms, specifically in this case deployed on Amazon Web Services (AWS) Elastic Kubernetes Service (EKS) platform. This research can be extended to various other cloud platforms including Google Cloud Platform (GCP), Microsoft Azure, IBM Cloud, and so on, utilizing similar sorts of native cloud tools. In order to achieve this we deployed an open-source “Sock-Shop” microservice application on the AWS EKS cluster. Simulated cybersecurity attacks and subsequently collect logs using AWS Cloudwatch. The log data obtained serve as the dataset for training and evaluating deep learning models such that it can be utilized to predict the type of cyber attacks conducted. This thesis systematically monitors and analyzes cyber attacks in microservice applications using AWS services and Deep Learning techniques. This research contributes to enhancing the security of the newly adapted Microservice architecture and also opens avenues for future work in advancing attack detection and prevention mechanisms.

Chapter 1 - Introduction

1.1 Background and Motivation

With the increase in complexity of software systems and the increase in demand for scalable and flexible software architectures the adoption of microservice architectures has been widespread. Traditional monolithic architecture has been around since the beginning, where a singular, large computing network with one code base interconnects all of the business concerns together. In contrast, microservice-based applications consist of various loosely-connected, independent services which can be deployed independently and interact with each other over a communication protocol (like RESTful APIs or message queues). This approach has some edge over the traditional monolithic architecture like improved scalability, maintainability, and resilience, which is why it might be a better choice for organizations who want to keep up with the rapidly evolving software market.

Due to this nature of microservice architecture, new security concerns have arisen which need to be addressed to ensure the integrity of these newer systems. This design has increased number of components and communication channels, which can be significantly vulnerable to cyber attacks. The communication between numerous services can be exploited and intercepted by targeting individual services. As a result, there is a need to develop effective methodologies in order to monitor, analyze and detect these attacks in microservice applications.

As the applications are progressing towards this newer architecture, cloud service providers like Amazon Web Services (AWS) have been developing tools to efficiently support this transition. AWS is one of the leading cloud service providers which offers a wide range of tools and services to facilitate this architecture in terms of scaling and maintainability. One of the services that go hand in hand with this architecture is Elastic Kubernetes Service (EKS).

EKS simplifies the deployment and management of containerized applications so that the developers can focus more on the core functionality and business logic of their applications. But still, the developers are responsible for keeping the security measures in place to ensure the integrity of the application, which directly raises the question about the importance of continuously monitoring and analyzing the security landscape.

We are already aware of the machine learning capabilities for classifying cyber security attacks based on the pattern found in network data. Using Deep Learning models on the logs collected from the microservice traffic can be a good choice to predict possible attacks in these microservice applications and thus improve the security of these systems.

1.2 Problem Statement

With the rise in adoption of the Microservice architecture concerns about security is a natural thing to arise. Until now we had to deal with security issues pertaining to the Monolithic architecture, with the rise of Machine Learning technologies leading to some major advancements in the field of cyber security we have witnessed the development of numerous AI-based security tools. Now challenges arise in the new microservice architecture system in terms of integrity, confidentiality, and availability due to the fact of it being complex and having a greater surface area susceptible to attacks. With the rapid growth of this architecture, there is a need to develop methodologies to effectively detect, classify and analyze cybersecurity attacks on microservice applications.

The problem addressed in this thesis has two halves:

The first one is implementing a method for simulating and monitoring cybersecurity attacks on a microservice application running on the AWS EKS platform. This involves deploying an example application, altering its interface to allow various cyberattacks, then collecting relevant logs using AWS CloudWatch.

And secondly, applying deep learning techniques to analyze log data and predict types of cyberattacks against a microservice application requires preprocessing and feature extraction from the logs, selecting suitable deep learning models, and assessing their accuracy in recognizing and classifying attacks.

By addressing these problems, this thesis aims to contribute to the improvement in security aspects of the microservice architecture. The outcome of this research has the potential to influence the development of more advanced security measures pertaining to microservice applications deployed in cloud environments.

1.3 Objectives and Scope

The primary focus of this research is to develop a systematic approach in order to monitor and analyze cyber security attacks in microservice applications which have been deployed to the AWS cloud platform. Following are the objectives that have been identified to achieve the goal:

1. Deploying a representative microservice application, “sock-shop” in this case on the AWS EKS cluster, to simulate a real-world e-commerce application.
2. Modify the “sock-shop” application’s interface in order to carry out the execution of cybersecurity attacks, which can simulate real-world threat scenarios.
3. Obtain the log data from the Cloudwatch service and preprocess it to extract meaningful data so that it can be fed to the Deep Learning models for training and evaluation.
4. Select and implement an appropriate Deep Learning model for detecting and classifying cyber attacks on microservices.
5. Evaluating the performance of implemented Deep Learning models.
6. Identify challenges, limitations, and potential improvements to the proposed research.

Also exploring future research directions in the domain of microservice application security.

The scope of this thesis is limited and focused on the deployment and monitoring of the "sock-shop" microservice application on the AWS EKS platform and using deep learning models for log analysis. Although its primary focus is microservice applications, its findings, and methodologies could also apply to other distributed systems. Unfortunately, it does not cover countermeasures or remediation strategies for detected cyberattacks; additionally, this research only analyzes logs collected through AWS CloudWatch without exploring other monitoring or logging solutions available.

1.4 Thesis Organization

The thesis has been organized into 5 chapters, covering different aspects of the research. Here is the given structure:

1. Introduction: This section provides an overview, and highlights the background, motivation, problem statement, objectives, scope, and limitations. This lays out the context and outlines the research questions that this thesis addresses.
2. Literature Review: Presents the review of the related and relevant literature. Essential topics such as microservice architecture, cyber security attacks, AWS Services such as EKS and Cloudwatch, and the application of Deep Learning for detecting security attacks are covered. This provides a foundation for the proposed research methodology.
3. Methodology: This section lays out the details of the research methodology that was executed. It can explain the process of deploying the "Sock-Shop" microservice application on AWS EKS, modifying its interface, simulating attacks, and log collection using Cloudwatch, etc.
4. Results: Discusses the implementation of the proposed methodology, system

architecture, attacks scenarios, datasets, and deep learning models used in the study. It concludes with a discussion of challenges, lessons learned, and potential improvements to the approach.

5. Conclusion and Future Work: Provides a summary of findings and outcomes from this research. It also depicts the limitations of the research and suggests future research directions.

Chapter 2 - Literature Review

2.1 Microservice Architecture

Microservice architecture is a software development approach that focuses on building applications as a collection of small, independent, and loosely coupled services. Where each service has a specific functionality of its own (Newman, 2015). In contrast to the traditional monolithic style of architecture where all the components are tightly integrated and the application is deployed as a single unit. Microservices aim to provide advantages over monolithic architecture such as flexibility, modularity, and scalability. This is achieved by breaking down the complex application into smaller manageable services, hence the term microservices. These microservices can be developed, managed, and scaled independently.

These microservices are highly modular, self-contained, and resilient with each covering a certain responsibility (Richardson, 2018). Microservices communicate with each other using lightweight protocols such as RESTful APIs, gRPC, or Message Queues (Dragoni et al., 2017). This architecture enables application developers to be more agile, which in turn means faster development cycles, improved maintenance, and scalability.

Advantages that arise from adapting to Microservice architecture:

1. Scalability: Individual services can be scaled independently, allowing for more efficient resource utilization and performance optimization (Nadareishvili et al., 2016).
2. Flexibility: Microservices can be developed and deployed using different technologies, programming languages, or frameworks, allowing teams to choose the best tools for their specific needs (Lewis & Fowler, 2014).
3. Faster Time-to-Market: Smaller, independent services enable faster development and deployment cycles, reducing the time it takes to bring new features or improvements to market (Balalaie et al., 2016).

4. Fault tolerance: The failure of a single microservice is less likely to cause a complete application outage, as other services can continue to function independently (Villamizar et al., 2016).

Due to all these advantages over traditional architecture, several security concerns arise. Firstly due to the distributed nature of services, each service needs to establish a communication protocol with other services which increases the attack's surface as each service has a set of its own vulnerabilities which can be potential entry for attackers (Edmonds, 2016). Even more, the logging and monitoring part can get more tricky in a distributed environment because of the aggregation and correlation of data from multiple services (Haselmann & Schmid, 2019).

2.2 Cybersecurity attacks on Microservices

Microservices are vulnerable to various cybersecurity attacks due to their distributed nature. This can lead to complex interdependencies and reliance on insecure communication channels between individual services (Edmonds, 2016; Zdankiewicz & Skrzypczak, 2019).

Some of the common cybersecurity attacks include

1. Distributed denial of service (DDoS) attacks: Overwhelming a service with traffic to render it unavailable.
2. Injection attacks: Exploiting vulnerabilities in APIs or data stores to inject malicious code or commands.
3. Data breaches: Unauthorized access to sensitive data stored or transmitted by microservices.
4. Man-in-the-middle attacks: Intercepting and potentially altering the communication between services.

2.3 Monitoring and Logging in Microservices

The maintenance of any software system in terms of security, performance, and reliability requires monitoring and logging. Microservices being of distributed nature, their monitoring and logging requirements differ from traditional monolithic architectures.

Monitoring and logging provide invaluable insight into the behavior of distributed services and how they communicate with one another, helping detect possible security threats. Log data collected can be utilized for identifying performance bottlenecks, diagnosing failures, and pinpointing security incidents (Haselmann & Schmid, 2019).

Logging is the practice of recording events, errors and other relevant data generated by microservices during execution. We can use these logs to identify issues, diagnose failures and detect security incidents. Generally, log data includes request/response info, user actions, error messages (like HTTP errors) and system events (Turnbull, 2014).

Monitoring is the practice of collecting and analyzing system metrics such as performance, health, and resource usage by an application. Key metrics include response times, throughput, error rates, CPU, Memory, and Network utilization. Monitoring tools like Datadog, Prometheus and AWS Cloudwatch are examples; these can be employed to collect metrics and generate alerts based on predefined thresholds (Turnbull, 2014).

Analyzing log data can help detect patterns, trends and anomalies that could indicate performance issues or security incidents. Anomaly detection techniques such as statistical methods, machine learning and deep learning can be applied to log data to detect unusual behavior or potential threats (Buczak & Guven, 2016). Visualization tools like Kibana or Grafana also facilitate this analysis by providing graphical representations of trends, patterns and anomalies.

2.4 AWS EKS and CloudWatch

AWS CloudWatch is a monitoring and observability service that provides users with data to track applications, adjust system-wide performance changes, optimize resource utilization, and gain an overall view of operational health. Collecting various metrics, logs, events so users can set alarms, visualize logs, and take automated actions based on predefined conditions.

CloudWatch's integration with AWS services such as EKS makes it ideal for monitoring a "sock-shop" microservice application deployed on an EKS cluster. Through CloudWatch, this thesis seeks to collect logs and record events related to cybersecurity attacks against the microservice application.

This thesis necessitates the integration of EKS and CloudWatch, to facilitate efficient collection and analysis of logs generated during cyber attacks on a "sock-shop" microservice application. EKS manages application deployment and operations while CloudWatch keeps an eye on performance metrics and collects logs.

To enable log collection, it is necessary to configure the EKS cluster and microservices running within it to forward logs onto CloudWatch. This involves setting up Kubernetes namespaces, creating log groups in CloudWatch, and configuring container insights within EKS. Once these steps have been taken, any logs generated during cyber attacks can be collected, stored, and analyzed using CloudWatch.

Integration of EKS and CloudWatch not only simplifies log collection and monitoring, but provides a scalable and robust solution for detecting and classifying cyber security attacks in microservice applications. In the following sections of this thesis, the collected logs will be fed into deep learning models that will predict and classify the type of attack.

2.5 Deep learning models for attack detection and classification

2.5.2 Deep Learning

Deep learning, a subfield of machine learning, utilizes artificial neural networks to model complex patterns and representations in large datasets. Deep learning models are capable of automatically extracting features from raw data without the need for manual feature engineering. These models have shown remarkable success across various domains such as natural language processing, computer vision, and speech recognition.

2.5.2 Application of Deep Learning in Cybersecurity

Deep learning can be applied to many different domains to assist cybersecurity operations and to boost cybersecurity performance. Deep learning techniques have seen an exponential rise in recent years due to their capacity for detecting novel and sophisticated cyber threats. Deep learning models are used for intrusion detection, malware classification, anomaly detection - just to name a few. By analyzing network traffic logs or application data with these models, security professionals can better detect patterns indicative of malicious activities and take more effective responses when confronted with such hazards.

2.5.3. Deep Learning Models for Cybersecurity Attack Detection

Many deep learning models have been proposed and applied to cybersecurity attack detection. Some of the most widely utilized ones include:

Convolutional Neural Networks (CNNs): CNNs are particularly skilled at processing grid-like data, such as images or time series records. In cybersecurity applications, CNNs have been used to analyze network traffic and log files for intrusion detection and anomaly detection.

Recurrent Neural Networks (RNNs): RNNs are designed to handle sequential data, making them ideal for analyzing time-series or sequences of events in cybersecurity. Long Short Term Memory (LSTM) networks - variants of RNNs - have been employed in detection of intrusions and anomalies in network traffic and system logs.

Autoencoders: Autoencoders are unsupervised deep learning models that can learn efficient data encodings by reconstructing input data. They have been applied to anomaly detection tasks in cybersecurity, detecting unusual patterns in network traffic or logs that could indicate an attack.

Deep Belief Networks (DBNs): DBNs are generative models composed of multiple layers of restricted Boltzmann machines. They have been employed for intrusion detection and malware classification tasks in cybersecurity.

2.5.4. Challenges and Prospects

Though deep learning models show promise in detecting cybersecurity attacks, there are several challenges and considerations when applying them to real-world scenarios:

Data Quality and Preprocessing: Deep learning models rely heavily on the quality and representation of input data to train their models and detect attacks effectively. Ensuring your data is clean, relevant, and appropriately preprocessed is paramount for successful model training and detection.

Model Interpretability: Deep learning models often get labeled "black boxes" due to their intricate internal structures. Analyzing and explaining the model's decisions may prove challenging, potentially hindering its adoption in cybersecurity applications.

Computational Resources: Training deep learning models necessitates a considerable amount of computational power, such as powerful hardware and parallel processing. This demand may pose an obstacle for organizations with limited infrastructure.

Adversarial Attacks: Deep learning models are vulnerable to adversarial attacks, in which an attacker creates inputs designed to cause the model to make incorrect predictions. Protecting deep learning models against such assaults is currently a research focus in cybersecurity.

2.5.5. Concluding Remarks to the Thesis

This thesis proposes to use deep learning models to detect and classify cyber attacks on a "sock-shop" microservice application deployed on AWS EKS. Logs collected from AWS CloudWatch will serve as input data for these models. Selecting an appropriate deep learning model while taking into account its challenges and limitations in cybersecurity will be key for successfully executing this proposed attack detection framework.

2.6 Related Works

Studies have explored the use of cloud services like AWS for monitoring and logging security-related activity. Almorsy et al. (2016) proposed a framework to integrate security monitoring and logging in cloud environments using AWS services like CloudWatch, Lambda, and Elasticsearch; their study demonstrated that this framework could enhance detection rates for security incidents within cloud-based setups. Alharkan et al. (2018) also conducted research using AWS CloudWatch as an intrusion detection platform in cloud environments; they demonstrated how it could effectively detect cyber attacks as well as anomalies within these networks.

Studies have employed deep learning techniques for detecting cybersecurity attacks. For instance, Yin et al. (2017) proposed a deep learning-based intrusion detection system using CNNs and LSTMs to analyze network traffic; their approach proved more successful at detecting intrusions than traditional machine learning methods.

In conclusion, the literature indicates a growing interest in detecting cyber attacks within microservice applications, using cloud services for monitoring and logging, as well as applying deep learning techniques for attack detection. This thesis builds upon these studies by proposing an

innovative framework that utilizes AWS EKS, CloudWatch, and deep learning models to monitor and analyze cyber attacks within a microservice application.

Chapter 3 – Methodology

In this chapter, we aim to monitor and analyze cybersecurity attacks in microservice applications using Amazon Web Services (AWS) by deploying a "sock-shop" microservice application to an AWS Elastic Kubernetes Service (EKS) cluster.

3.1 Deploying the "Sock-Shop" microservice application to AWS EKS

In this section, we describe the process of deploying the "sock-shop" microservice application to an AWS Elastic Kubernetes Service (EKS) cluster. This involves configuring the AWS EKS cluster, configuring its infrastructure, and deploying the "sock-shop" application. But first, “What is Sock-Shop?”. Sock-Shop is an open source Microservice based application simulating an e-commerce platform that sells socks. It is developed and maintained by *Weaveworks*. It consists of multiple microservices where each service handles a specific aspect of the application such as: Front End, Carts, Payment, Catalogue, User(backend service), and so on. It has a polyglot architecture, which basically means every service is independent in terms of language and stack uses, it utilizes different languages, frameworks and databases for each microservice. For instance, Cart service utilizes Java+MongoDB, Catalogue service utilizes Go+MySQL and so on. The purpose of choosing this application in our thesis was to serve as a real-world microservice application which would act as a foundation for deploying, monitoring, and analysing attacks in the cloud environment.

3.1.1. AWS EKS Configuration and Setup

Firstly an AWS account with root user permissions or administrator access is required. From here onwards all deployment tasks were carried out under an IAM user that has to have the necessary permissions to carry out deployment and configuration tasks. In our case, an IAM user under the name *eksuser* was created, which has an Administrator Access permissions policy attached. Now a custom policy needs to be generated that grants access to EKS

resources. The policy is named *eksadmin* which looks like this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "eks:*"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": "eks.amazonaws.com"
        }
      }
    }
  ]
}
```

Figure 1. eksadmin policy

A new user group is created called *eksgroup* and the above created policy *eksadmin* is attached to this group. Now we can attach this group to the user *eksuser* that was initially created. We could have directly attached the policies to the user but it is a good practice to create groups and attach users to that certain group containing the required policies.

We now login with *eksuser* credentials, and create a *Cloud9* IDE environment. AWS Cloud9 is an integrated IDE that in our case lets us communicate with AWS services that we want to configure. Now we need to create a Role, that has an Administrator Access policy attached. Then we look for our Cloud9 EC2 instance and replace the existing IAM role with the newly created Admin Access Role. This enables our Cloud9 IDE to have access to every service within our AWS account. Now we need to turn off AWS-managed temporary credentials in the Cloud9 terminal settings. In order to check if the IAM role is valid we used the following command:

```
# Validate that our IAM role is valid.
aws sts get-caller-identity --query Arn | grep partnerName-
workshop-admin -q && echo "IAM role valid" || echo "IAM role NOT
valid"
```

Figure 2. IAM Role Validation

Note: that throughout the deployment and configuration, the AWS location had been set to *us-west-2 (Oregon)*.

We can now install the tools required for configuration and deployment from the Cloud9 terminal directly. Some important CLI tools we need for the deployment are *awscli*, *eksctl*, and *kubect*. *eksctl* is used to create clusters on EKS and *kubect* allows us to run commands against Kubernetes Clusters.

Now we can create a cluster called *sockshop-eks-cluster* using *eksctl*. Here is what the command looks like;

```
eksctl create cluster --name my-cluster --region region-code
```

Figure 3. Create Cluster

Before moving on further with the deployment, we need to create Cluster Role and Cluster Role Binding. Cluster Role and Cluster Role Binding are essential concepts in Kubernetes that govern access control and permissions within a cluster. They form part of the Role-Based Access Control (RBAC) system, which helps manage and restrict user or service access to resources within the cluster. Let us examine these two terms more closely:

Cluster Role: A Cluster Role is a set of rules that determines the permissions for various resources within a Kubernetes cluster. They specify what actions (like "get," "list," "create," "update," "delete") can be performed on which resources (e.g., pods," services," deployments") within this cluster. Cluster Roles are independent from any particular namespace so they apply uniformly across all resources without exception.

Cluster Role Binding: A Cluster Role Binding is a Kubernetes resource that links a Cluster Role to an individual, group, or service account. By doing so, it grants the permissions defined in the Role to those specified subjects across all namespaces within the cluster - rather than just one.

After creating both the Cluster Role and Cluster Role Binding configurations we need to edit

aws-auth ConfigMap. We need to assign the particular group that was mentioned in the Cluster Role and Cluster RoleBinding config files, to the eksuser inside the ConfigMap of aws-auth. Here is the command to edit the ConfigMap:

```
$ kubectl edit configmap aws-auth -n kube-system
```

Inside of the ConfigMap we add mapUsers entry to the data, which in our case looks like this:

```
mapUsers: |
  - userarn: arn:aws:iam::868695241754:user/eksuser
    username: eksuser
    groups:
      - system:masters
      - system:nodes
      - eks-console-dashboard-full-access-group
```

Figure 4. ConfigMap mapUsers

Also change:

system:bootstrappers to system:masters

Now we can grab kubeconfig from AWS:

```
aws eks --region us-west-2 update-kubeconfig --name sockshop-eks-cluster
```

Figure 5. KubeConfig

We can now successfully be able to view nodes: kubectl get nodes. This was all the configuration we needed to do before deploying the Sock-Shop application to the cluster.

3.1.2. Deployment of "Sock-Shop" to the Cluster

In this section, we shall deploy the Sock-Shop microservice to our configured EKS cluster,

Firstly we clone the GitHub repo using our Cloud9 console via:

```
git clone https://github.com/microservices-demo/microservices-demo
```

Figure 6. Clone Sock-Shop

Creating the namespace:

```
kubectl create namespace sock-shop
```

Figure 7. Create Namespace

Navigate to the Kubernetes manifest file which could be found under deploy/kubernetes folder.

After verifying the manifest file we finally deploy the application using the command:

```
kubectl apply -f complete-demo.yaml
```

Figure 8. Apply Kubernetes Manifest

Front end service can now be exposed via a Load Balancer:

```
kubectl expose service front-end --name=front-end-lb --port=80 --target-port=8079  
--type=LoadBalancer --namespace=sock-shop
```

We can now navigate to our Load Balancer instance and access the Sock-Shop application via the DNS name of the given Load Balancer.

3.2 Modifying the "sock-shop" interface for cybersecurity attacks

In order to modify our “sock-shop” application to be able to perform a Brute Force attacks we had to make some changes to the front-end interface. The actual login page is embedded into a Modal which caused hindrance in performing Brute Force attack. Thus the Login page had to be separated into a new web page so that we are able to perform the attacks effectively. As this is a microservice application, the front-end is handled by a microservice called “front-end”. So in order to make changes we had to modify the Docker images, make the necessary changes to it, reupload the image to our own docker hub and finally modify the Kubernetes manifest files. Here is the detailed instruction of how it was done: To modify the files in a Docker image, you'll need to follow these steps:

Get the Dockerfile and source code for the image:

```
git clone https://github.com/microservices-demo/front-end.git
```

Figure 9. Clone Front-End

Navigate to the directory where the HTML files are located. You'll find them in the 'public' folder or another folder containing the application's code. Make the necessary changes to the HTML and JavaScript files you want to modify. Now, navigate to the root folder of the repository (where the Dockerfile is located), and build the Docker image with a new tag using the following command:

```
docker build -t your_dockerhub_username/modified_front-end:0.3.12 .
```

Figure 10. Build Front-End

Replace your_dockerhub_username with your Docker Hub username.

Push the modified image to your Docker Hub account with the following command:

```
docker push your_dockerhub_username/modified_front-end:0.3.12
```

Figure 11. Push Front-End

To run the "sock-shop" application locally using your modified "front-end" image:

```
git clone https://github.com/microservices-demo/microservices-demo.git
```

Figure 12. Clone Sock-Shop

Modify the "docker-compose.yml" file:

Navigate to the "microservices-demo/deploy/docker-compose" directory, and open the "docker-compose.yml" file in a text editor. Find the "front-end" service definition, which should look something like this:

```

services:
  front-end:
    image: jahanzaibmalik75/modified_front-end:0.3.13
    hostname: front-end
    restart: always
    cap_drop:
      - all
    read_only: true
  edge-router:
    image: weaveworksdemos/edge-router:0.1.1
    ports:
      - '80:80'
      - '8080:8080'
    cap_drop:
      - all
    cap_add:
      - NET_BIND_SERVICE
      - CHOWN
      - SETGID
      - SETUID
      - DAC_OVERRIDE
    read_only: true
    tmpfs:
      - /var/run:rw,noexec,nosuid
    hostname: edge-router
    restart: always

```

Figure 13. Docker Compose File

Replace the image property value with modified image's name:

image: jahanzaibmalik75/modified_front-end:0.3.13

Navigate to the "microservices-demo/deploy/docker-compose" directory in your terminal and run the following command:

docker-compose up

This command will start all the services defined in the "docker-compose.yml" file, including the modified "front-end" service.

Replacing the front-end service with modified front-end service in an existing EKS cluster:

First, locate the Kubernetes manifest file (YAML) for the "sock-shop" application that we used to deploy it on the EKS cluster. We can find the sample manifest in the "microservices-demo" repository:

<https://github.com/microservices-demo/microservices-demo/blob/master/complete-demo.yaml>

Figure 14. Kubernetes Manifest Download

Open the manifest file in a text editor and find the Deployment definition for the "front-end" service. Look for a section that looks like this:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: front-end
  namespace: sock-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      name: front-end
  template:
    metadata:
      labels:
        name: front-end
    spec:
      containers:
      - name: front-end
        image: weaveworksdemos/front-end:0.3.12
        resources:
          limits:
            cpu: 300m
            memory: 100Mi
          requests:
            cpu: 100m
            memory: 300Mi
        ports:
        - containerPort: 8079
```

Figure 15. Kubernetes Manifest

Replace the image property value with modified image's name:

image: jahanzaibmalik75/modified_front-end:0.3.12

Apply the updated Kubernetes manifest:

Save the changes to the manifest file and then apply the updated manifest to EKS cluster by running the following command:

kubectl apply -f complete-demo.yaml

Now, the EKS cluster is running the "sock-shop" application with the modified front-end service.

In our case, we modified the Login interface, in the original application the Login functionality was embedded into a modal on the homepage which was not optimal to perform the attacks like

brute force. So embedded a whole new login page to fix that issue.

3.3 Attack generation and simulation

To perform Brute attack application called BurpSuite was used. BurpSuite is a application security testing platform which has tools to perform controlled security attacks. The application had been modified to contain injection points and vulnerable components in the previous section. BurpSuite includes these components:

Scanner: A vulnerability scanning application tool.

Proxy: An intercepting too that allows user to intercept and manipulate HTTP request and responses.

Intruder: Tool to automate automate custom attacks such as brute force, parameter fuzzing etc.

Repeater: Tool to manipulate individual HTTP requests.

The first step was to configure BurpSuite to work with the 'sock-shop' application, which was done by setting up a BurpSuite Proxy between our local machine and 'sock-shop' on the EKS cluster. The local Browser was configured to use the Burpsuite Proxy. Then the endpoints of our application were analyzed to understand them better.

Targeting our application such that the attacks could be performed included using BurpSuite included:

Brute Force attack: Same tool Burp Intruder could be used, where Intruders attack configurations could be employed to perform a brute force attack.

Brute Force attack Simulation Process: This attack was performed on the application Login Page.

Password Data:

Utilized *RockYou.txt* - a widely known password dictionary file, which was used with BurpSuite as a Payload.

Modifying RockYou.txt password file:

Before feeding the password file(payload) to the intruder it was modified to following format:

Base64{username:password}. This was achieved using a Python script which converted the dictionary file into the encoded format. This had to be done because of the authorization enforced by the “Sock-Shop” application while sending data over HTTP.

```
pass_encode.py
1  import base64
2
3  input_file = 'rockyou.txt'
4  output_file = 'rockyou_encoded.txt'
5
6  # Set the username variable
7  username = 'christopher'
8
9  # Set the limit for the number of entries to process
10 entry_limit = 300
11 current_entry = 0
12
13 with open(input_file, 'r') as infile, open(output_file, 'w') as outfile:
14     for line in infile:
15         # Stop processing when the entry limit is reached
16         if current_entry >= entry_limit:
17             break
18
19         password = line.strip()
20         modified_password = f"{username}:{password}"
21         base64_encoded_password = base64.b64encode(modified_password.encode('utf-8')).decode('utf-8')
22         outfile.write(base64_encoded_password + '\n')
23
24         # Increment the entry count
25         current_entry += 1
```

Figure 16. Python Encoding Script

The Brute Force attack was carried for 2 different payloads, first with 5000 samples and second with 300 samples. As the attacks were carried out, logs and metrics were collected into our CloudWatch which had been setup prior to this. The attack logs gave us insights about what possible types of attacks were performed.

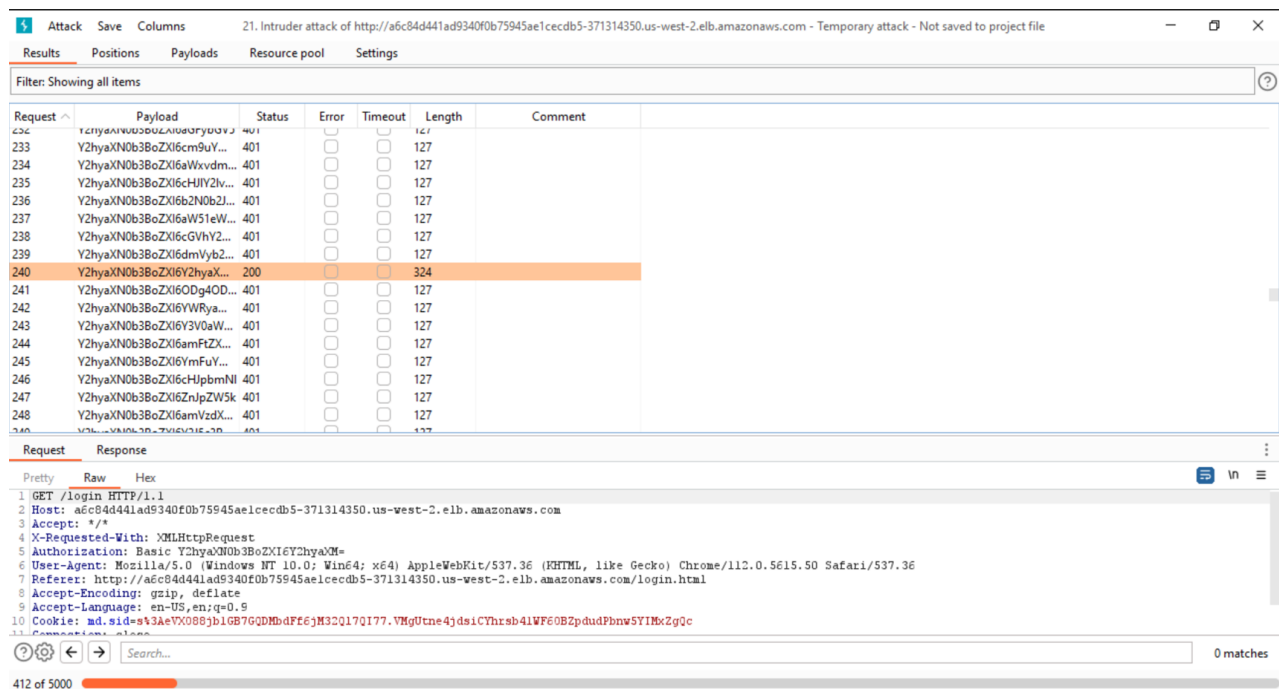


Figure 17. BurpSuite

In addition to BurpSuite, an application call JMeter was used to carry out DDoS attack. JMeter is a popular load testing application used to test the applications in a controlled manner. JMeter was configured for various loads by configuring metrics such as Number of Threads, Ramp up Time, Loop Count and Duration.

DDoS Attack simulation process: This attack was performed on the application Home Page, basically the front end of our application which was exposed via a load balancer.

3 different test loads were performed to simulate real world attack scenarios:

- Moderate Load
- High Load
- Exterme Load

Moderate Load:

Number of threads=100

Ramp Up=120

Loop Count=10

Duration=1200 seconds

High Load:

Number of threads=500

Ramp Up=300

Loop Count=20

Duration=3600 seconds

Extreme Load:

Number of threads=1000

Ramp Up=600

Loop Count=50

Duration=7200 seconds

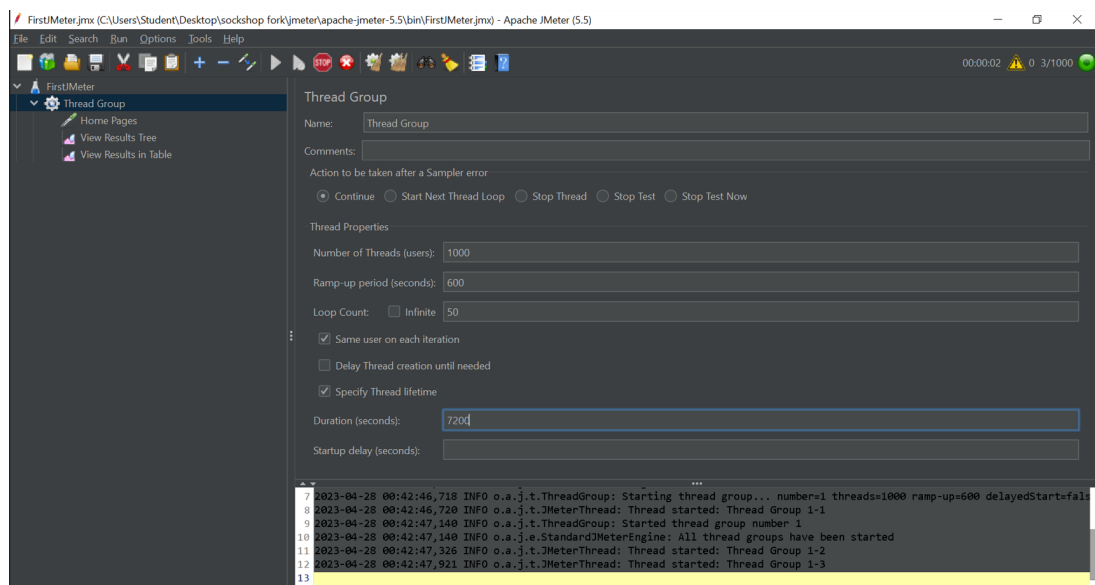


Figure 18. JMeter Setup

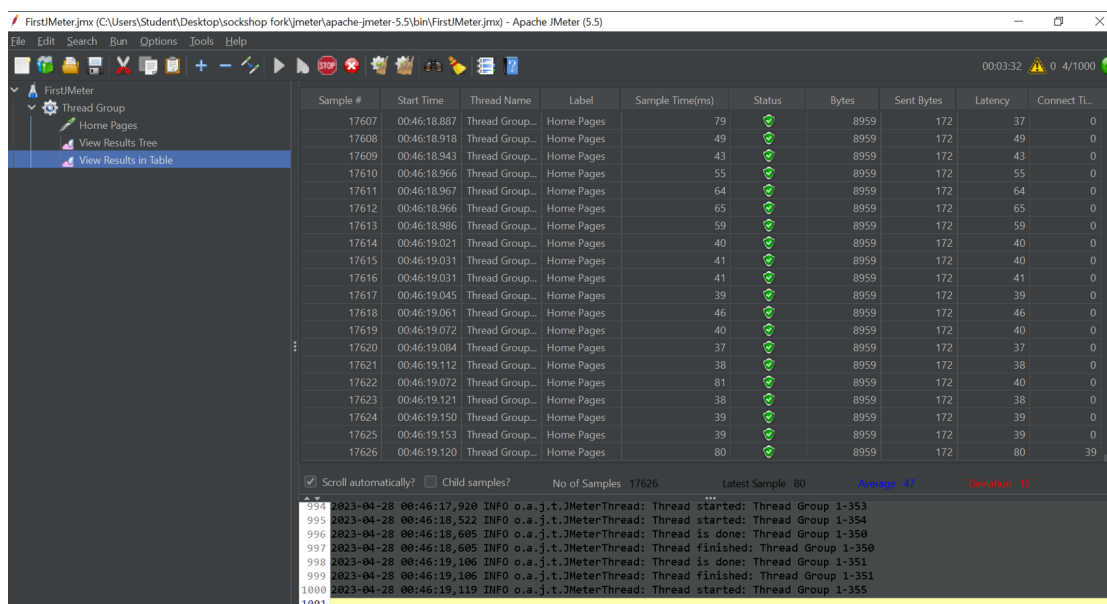


Figure 19. JMeter Load Test

3.4 Collecting logs using AWS CloudWatch and Preprocessing

In this section we will discuss how we configured our AWS Cloudwatch console and application to be able to collect logs generated during the attack simulations from our sock-shop application. We used an opensource log processor called FluentBit, which processes and forwards the logs which can be then collected in CloudWatch.

Firstly we had to deploy a FluentBit Daemon set, which had to run on each individual worker node so that logs can be collected from all the containers running on the node. Configure FluentBit to collect from our container runtimes and forward logs to AWS CloudWatch by setting up output plugin in the FluentBit configuration. We used standard logging libraries and output formats to make sure compatibility issues don't arise with FluentBit and Cloudwatch.

AWS CloudWatch Container Insights:

A monitoring and analytics service for containers and microservices. It is Integrated with Amazon EKS and Kubernetes. This service Collects logs, metrics, and events from the "sock-shop" microservices. This facilitated effective monitoring and analysis of cybersecurity attacks.

Configure and enable Container Insights in the EKS cluster:

Attaching the necessary policy to the IAM role for your worker nodes

Access one of our worker nodes running our cluster via EC2 console. Select the worker and identify the IAM policies attached, modify the policy to include:

1. CloudWatchAgentServerPolicy
2. CloudWatchLogsFullAccess

Finally attaching these policies.

To deploy Container Insights using the quick start:

```
ClusterName=<my-cluster-name>
RegionName=<my-cluster-region>
FluentBitHttpPort='2020'
FluentBitReadFromHead='Off'
[[ ${FluentBitReadFromHead} = 'On' ]] && FluentBitReadFromTail='Off' || FluentBitReadFromTail='On'
[[ -z ${FluentBitHttpPort} ]] && FluentBitHttpServer='Off' || FluentBitHttpServer='On'
curl https://raw.githubusercontent.com/aws-samples/amazon-cloudwatch-container-insights/latest/k8s-deployment-manifest-templates/deployment-mode/daemonset/container-insights-monitoring/quickstart/cwagent-fluent-bit-quickstart.yaml | sed
's/{{cluster_name}}/'${ClusterName}';s/{{region_name}}/'${RegionName}';s/{{http_server_toggle}}/'${FluentBitHttpServer}';s/{{http_server_port}}/'${FluentBitHttpPort}';s/{{read_from_head}}/'${FluentBitReadFromHead}';s/{{read_from_tail}}/'${FluentBitReadFromTail}' | kubectl apply -f -
```

In this command, *my-cluster-name* is the name of your Amazon EKS or Kubernetes cluster, and *my-cluster-region* is the name of the Region where the logs are published.

Figure 20. FluentBit Setup

Log Groups now deployed onto Amazon Cloudwatch - Service Wise. Each service has its own Log Group.

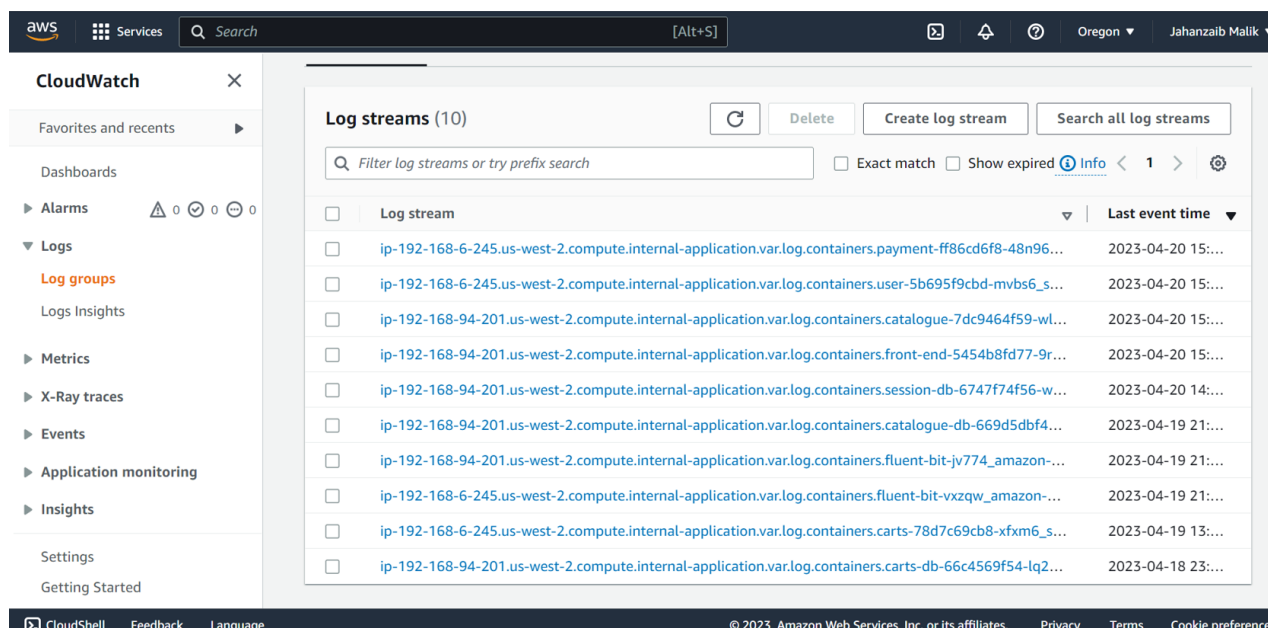


Figure 21. Log Groups

Visualizing and Collecting Data via Container Insights:

Sock-Shop cluster is visible in Container Insights and Each service has its own metric dashboard eg. user, front-end, carts etc.

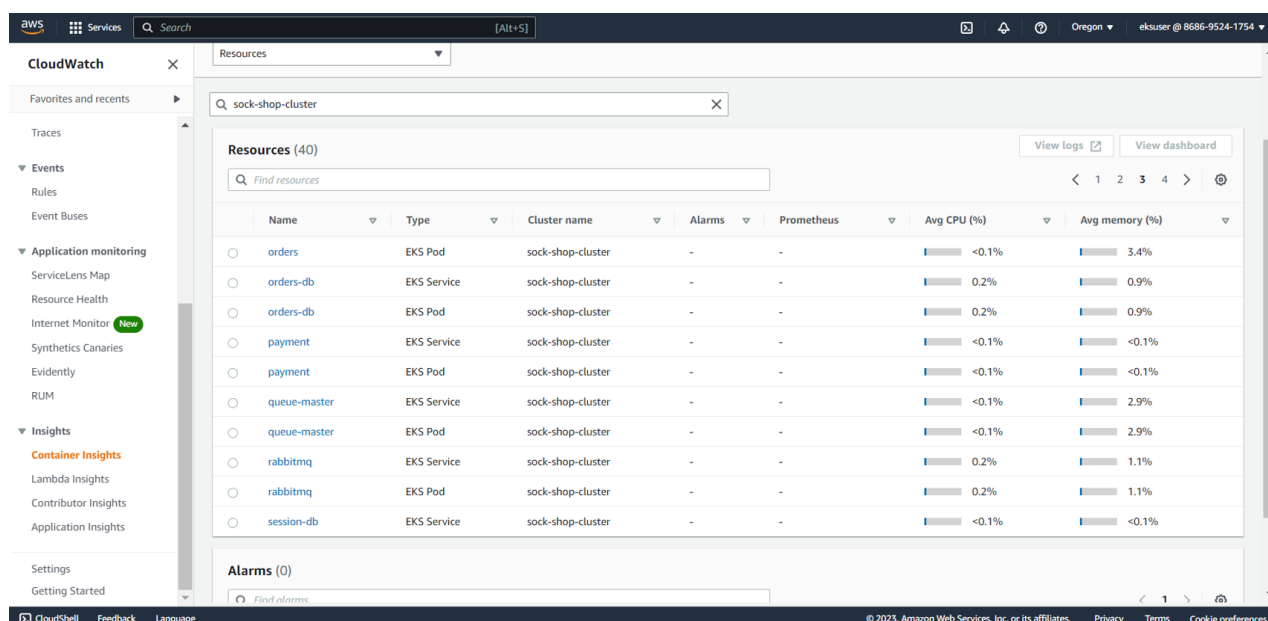


Figure 22. Container Insights

Now we can collect logs for different services running on our cluster. Here we collected data from

metrics such as:

- CPU Utilization
- Memory Utilization
- Network
- Pod CPU Utilization
- Pod Memory Utilization

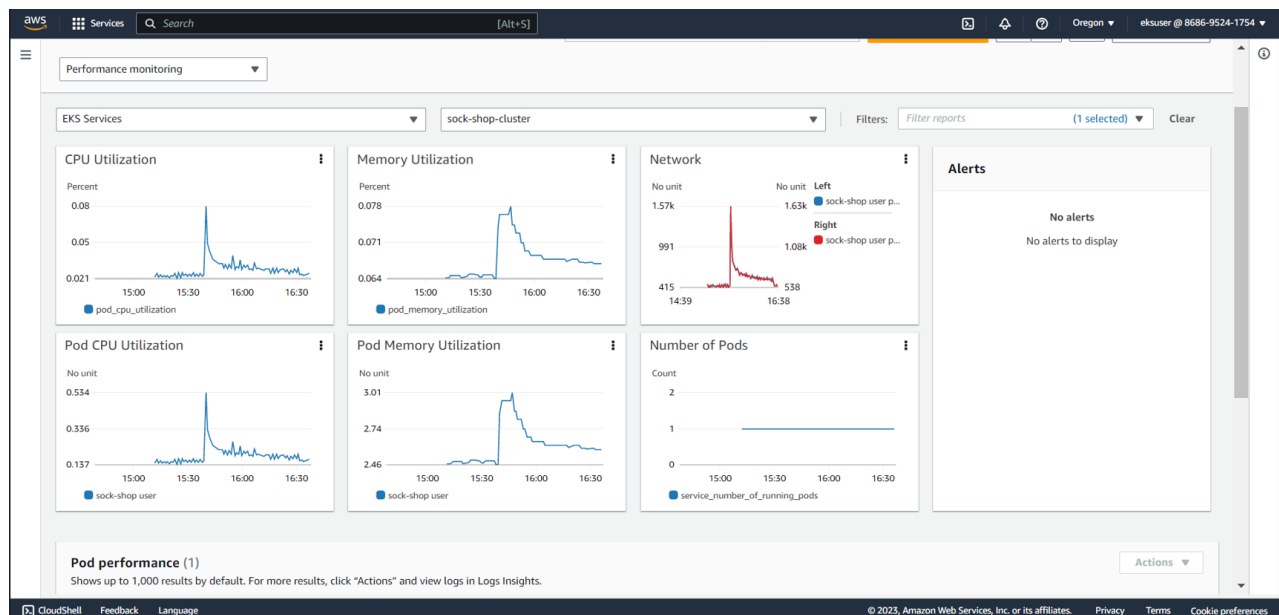


Figure 23. Container Insights - User Service Metrics

Log files are Time Indexed. For DDoS attack, 5 separate metrics were collected from the front-end-lb service as the attack was performed on the Homepage of the application.

Same for BruteForce attack, 5 separate metrics were collected from the user(backend) service as the attack was performed on the Login Page of the application.

Collected Data Metrics/features:

- CPU Utilization
- Memory Utilization
- Network

- Pod CPU Utilization
- Pod Memory Utilization

During an attack on one service, the metrics for the other services such as, carts, catalogue etc. seemed to be unaffected. This was observed by comparing graph trends for attacked and non-attacked service.

The 5 different insight metrics were combined for a specific period for an individual attack for the specific service that was affected based on “timestamp” as the DataFrame index.

```
# Merge the DataFrames based on the timestamp index and add suffixes to overlapping columns
merged_df = (network_df.join(cpu_df, how='inner', lsuffix='_network')
              .join(memory_df, how='inner', lsuffix='_cpu', rsuffix='_memory')
              .join(pod_cpu_df, how='inner', lsuffix='_memory', rsuffix='_pod_cpu')
              .join(pod_memory_df, how='inner', lsuffix='_pod_cpu', rsuffix='_pod_memory'))
```

Figure 24. Merge Dataframe

All the data points were scaled using a MinMax Scaler and Normalized subsequently.

```
# Preprocess the data
data = pd.read_csv('Merged_Data.csv', parse_dates=['Timestamp'])
data = data.set_index('Timestamp')
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)
```

Figure 25. Scale and Normalize

```
In [2]: import pandas as pd
import os

# Set the working directory
data_folder = r"C:\Users\Student\Desktop\sockshop fork\Cloudwatch Insights Data\front-end-lb EKS Svc\12hr"
os.chdir(data_folder)

# Load each CSV file into a separate DataFrame
network_df = pd.read_csv('Network-2023_04_27_13_21_00-2023_04_28_01_19_00-UTC-7.csv', parse_dates=['Timestamp'])
cpu_df = pd.read_csv('CPU_Utilization-2023_04_27_13_21_00-2023_04_28_01_19_00-UTC-7.csv', parse_dates=['Timestamp'])
memory_df = pd.read_csv('Memory_Utilization-2023_04_27_13_21_00-2023_04_28_01_19_00-UTC-7.csv', parse_dates=['Timestamp'])
pod_cpu_df = pd.read_csv('Pod_CPU_Utilization-2023_04_27_13_21_00-2023_04_28_01_19_00-UTC-7.csv', parse_dates=['Timestamp'])
pod_memory_df = pd.read_csv('Pod_Memory_Utilization-2023_04_27_13_21_00-2023_04_28_01_19_00-UTC-7.csv', parse_dates=['Timestamp'])

# Set the timestamp as the DataFrame index
network_df = network_df.set_index('Timestamp')
cpu_df = cpu_df.set_index('Timestamp')
memory_df = memory_df.set_index('Timestamp')
pod_cpu_df = pod_cpu_df.set_index('Timestamp')
pod_memory_df = pod_memory_df.set_index('Timestamp')

# Merge the DataFrames based on the timestamp index and add suffixes to overlapping columns
merged_df = (network_df.join(cpu_df, how='inner', lsuffix='_network')
              .join(memory_df, how='inner', lsuffix='_cpu', rsuffix='_memory')
              .join(pod_cpu_df, how='inner', lsuffix='_memory', rsuffix='_pod_cpu')
              .join(pod_memory_df, how='inner', lsuffix='_pod_cpu', rsuffix='_pod_memory'))

# Export the merged DataFrame to a new CSV file
merged_df.to_csv('Merged_Data.csv')
```

Figure 26. Data Preprocessing

Attack Log Dataset(s) details:

Attack Type	Service	Data Collection Time Period	Sample Duration	Number of Features	Total number of samples
DDoS	front-end-lb	12 hours	1 minute	6	863
Brute Force	user	36 hours	5 minutes	6	718

Figure 27. Dataset(s) Details

3.5 Deep Learning Model Selection and Implementation

LSTM(Long Short-Term Memory):

Model Overview:

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) architecture. It is designed to handle time-series data and long-term dependencies effectively. In this project, the LSTM model is used for detecting DDoS and Brute Force attacks based on CloudWatch log data

LSTM Model Implementation:

Dataset was loaded into a dataframe with time as Index. The subsequent features were then scaled and normalized using MinMax Scaler. Sequences of log data were created with a fixed length (n=10) for model input. Data was then split into training and testing sets (80% training, 20% testing). LSTM model architecture was defined with 50 units and ReLU activation function. Model was trained using the 'adam' optimizer and mean squared error (MSE) loss function.

```
In [11]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import LSTM, Dense

# Preprocess the data
data = pd.read_csv('Merged_Data.csv', parse_dates=['Timestamp'])
data = data.set_index('Timestamp')
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

def create_sequences(data, n):
    X, y = [], []
    for i in range(len(data) - n):
        seq = data[i: i + n]
        X.append(seq[:-1])
        y.append(seq[-1])
    return np.array(X), np.array(y)

n = 10
X, y = create_sequences(scaled_data, n)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 28. LSTM Preprocessing

```
# Build the LSTM model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(X_train.shape[1], X_train.shape[2])))
model.add(Dense(y_train.shape[1]))
model.compile(optimizer='adam', loss='mse')

# Train the model
model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=1)

# Evaluate the model and make predictions
y_pred = model.predict(X_test)

# Calculate the mean squared error (MSE) between true and predicted values
mse = np.mean(np.square(y_test - y_pred))

# Set a threshold to classify instances as normal or DDoS attacks
threshold = 0.00005 # Adjust the threshold based on your understanding of the problem and the data

# Create a binary classification array (0: normal, 1: DDoS attack)
y_pred_class = np.where(np.square(y_test - y_pred) > threshold, 1, 0)
```

Figure 29. Building LSTM Model

Analysing the classification results using confusion matrix, precision, recall, and F-1 score.

```
: from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score

# Calculate the confusion matrix
cm = confusion_matrix(y_test.argmax(axis=1), y_pred_class.argmax(axis=1))

# Calculate precision, recall, and F1-score
precision = precision_score(y_test.argmax(axis=1), y_pred_class.argmax(axis=1), average='weighted')
recall = recall_score(y_test.argmax(axis=1), y_pred_class.argmax(axis=1), average='weighted')
f1 = f1_score(y_test.argmax(axis=1), y_pred_class.argmax(axis=1), average='weighted')

# Print the results
print("Confusion Matrix:")
print(cm)
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
```

Figure 30. Analysing LSTM

Calculating Macro and Micro weighted averages.

```
from sklearn.metrics import precision_recall_fscore_support

# Calculate precision, recall, and F1-score (micro, macro, and weighted averages)
precision_micro, recall_micro, f1_micro, _ = precision_recall_fscore_support(y_test.argmax(axis=1), y_pred_class.argmax(axis=1),
precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(y_test.argmax(axis=1), y_pred_class.argmax(axis=1),

# Print the results
print("Micro average:")
print(f"Precision: {precision_micro:.4f}")
print(f"Recall: {recall_micro:.4f}")
print(f"F1-score: {f1_micro:.4f}")

print("\nMacro average:")
print(f"Precision: {precision_macro:.4f}")
print(f"Recall: {recall_macro:.4f}")
print(f"F1-score: {f1_macro:.4f}")
```

Figure 31. Macro and Micro Averages LSTM

Autoencoder Deep Learning Model architecture:

Model Overview:

It is an unsupervised deep learning architecture used for representational learning. This neural network learns to efficiently compress and encode the data, and from that compressed data rebuild the original data back as accurately as possible. This deep learning model can be leveraged to detect anomalies. This is what we used as our next model for this thesis problem.

Autoencoder Model Implementation:

The data set was preprocessed the same way as for the LSTM model with an inclusion of adding labels, 0 signifying No Attack at a given time index and 1 signifying Attack was performed at the given time index. To be noted, labels were created only for validation and evaluation purpose not for training the model. Also the merged dataset was split onto 2 individual data sets:

1. Clean Data: Used to train the model.
2. Mixed Data: Used to validate and evaluate the model.

An Input layer was constructed with the same dimension as the preprocessed data, encoder layer with 32 neurons and ReLU activation. Decoder layer was the same dimension as the input layer

but sigmoid activation function instead.

Model training:

Optimizer: Adam

Loss function: Mean Squared Error (MSE)

Epochs: 100

Batch size: 32

Validation data: X_val

Reconstruction error calculation:

Calculated the mean squared error between the original test data (X_test) and the reconstructed test data (autoencoder.predict(X_test)).

Threshold selection for classification: Based on the 95th percentile of the reconstruction error

Adjusted the percentile value according to the problem requirements and data characteristics. The threshold was used to classify instances as normal or attacks (e.g., DDoS, Brute Force) by comparing the reconstruction error to the threshold value.

```
: import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

# Load the datasets
clean_data = pd.read_csv("Merged_Data - clean.csv", parse_dates=["Timestamp"]).set_index("Timestamp")
mixed_data = pd.read_csv("Merged_Data - mixed.csv", parse_dates=["Timestamp"]).set_index("Timestamp")

# Preprocess the datasets
scaler = MinMaxScaler(feature_range=(0, 1))
clean_data_scaled = scaler.fit_transform(clean_data.drop(columns="label"))
mixed_data_scaled = scaler.transform(mixed_data.drop(columns="label"))

# Split the mixed dataset into testing and validation sets
X_test, X_val, y_test, y_val = train_test_split(mixed_data_scaled, mixed_data["label"], test_size=0.2, random_state=42)
```

Figure 31. Autoencoder Preprocessing

```

import numpy as np
from keras.models import Model
from keras.layers import Input, Dense

input_dim = clean_data_scaled.shape[1]
encoding_dim = 32

# Define the autoencoder
input_layer = Input(shape=(input_dim,))
encoder = Dense(encoding_dim, activation="relu")(input_layer)
decoder = Dense(input_dim, activation="sigmoid")(encoder)

autoencoder = Model(inputs=input_layer, outputs=decoder)
autoencoder.compile(optimizer="adam", loss="mse")

# Train the autoencoder
autoencoder.fit(clean_data_scaled, clean_data_scaled, epochs=100, batch_size=32, shuffle=True, validation_data=(X_val, X_val))

```

Figure 32. Building Autoencoder

```

# Calculate the reconstruction error
reconstruction_error = np.mean(np.square(X_test - autoencoder.predict(X_test)), axis=1)

# Determine a threshold for classification
threshold = np.percentile(reconstruction_error, 95) # Adjust this percentile based on your understanding of the problem and the

```

Figure 33. Threshold Calculation

```

from sklearn.metrics import confusion_matrix, classification_report

# Classify instances as normal or anomalous
y_pred = np.where(reconstruction_error > threshold, 1, 0)

# Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Calculate precision, recall, and F1-score
report = classification_report(y_test, y_pred, target_names=["Normal", "Anomalous"])
print("Classification Report:\n", report)

```

Figure 34. Analyzing Autoencoder

```

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc

# Create a heatmap for the confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu", xticklabels=["Normal", "Anomalous"], yticklabels=["Normal", "Anomalous"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap")
plt.show()

# Calculate false positive rate, true positive rate, and the threshold for the ROC curve
fpr, tpr, thresholds = roc_curve(y_test, reconstruction_error)

# Calculate the AUC
roc_auc = auc(fpr, tpr)

# Plot the ROC curve
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="darkorange", lw=2, label="ROC curve (AUC = %0.2f)" % roc_auc)
plt.plot([0, 1], [0, 1], color="navy", lw=2, linestyle="--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC)")
plt.legend(loc="lower right")
plt.show()

```

Figure 35. Heatmap Generation

Autoencoder Deep Learning Model architecture with SMOTE:

Our data set seemed to be imbalance with minority of samples being the Attack samples. This resulted in an inaccurate recall. To overcome this issue we used SMOTE which stands for Synthetic Minority Over-sampling Technique. It is a technique by which we can oversample the minority class to better evaluate and validate our training model.

```

from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_mixed_resampled, y_mixed_resampled = smote.fit_resample(mixed_data_scaled, mixed_data["label"])

X_test, X_val, y_test, y_val = train_test_split(X_mixed_resampled, y_mixed_resampled, test_size=0.2, random_state=42)

autoencoder.fit(clean_data_scaled, clean_data_scaled, epochs=100, batch_size=32, shuffle=True, validation_data=(X_val, X_val))

```

Figure 36. SMOTE

Chapter 4 - Results

In this section we present details and results that were gathered from deploying our “sock-shop” application, executing attacks on it, collecting logs and training our Deep learning models for evaluation.

Firstly we went through deploying our application to the EKS cluster, which we went in detail in the section 3.1. Following deployment we looked at how the application was modified in section 3.2, in its interface so that necessary attacks could be executed for our testing.

Then we generated security attacks using a comprehensive tools called BurpSuite and JMeter in section 3.3. Attacks such as DDoS and Brute Force were performed on our application. These attacks simulated a real-world cyber threat which affect millions of applications on a daily basis. Then we ensured our logs are properly collected and documented in section 3.4. We used FluentBit as our log processor which send the logs from our application to the CloudWatch service. Using these logs we were able to gather and generate a dataset which then was used to train our deep learning models for evaluation.

The collected dataset was preprocessed to encode categorical features and parsing log entries. Using this data our deep learning model was trained for its ability to detect and classify cybersecurity attacks on the “sock-shop” application. Our Deep Learning Models included LSTM and Autoencoder.

The deep learning models were evaluated using a separate dataset, which had not been used during training. This ensured unbiased evaluation of our model performances.

The results obtained from this experiment suggest the proposed approach is effective in classifying real-world attack scenarios and potentially application owners can take necessary and immediate actions to mitigate the cause and progressive build better, resilient microservice applications. The

high accuracy, precision, recall, and F1-score achieved by the best performing model demonstrate its potential for real-world applications in monitoring and analyzing cyber threats in microservice-based environments.

LSTM Results:

DDoS: Confusion Matrix: <pre>[[0 0 0 0 0 0 0] [0 0 0 0 0 0 0] [0 0 0 0 0 0 0] [6 1 0 20 0 0 32] [1 0 0 0 0 0 0] [5 1 2 21 0 1 49] [2 1 0 0 0 0 0]]</pre>	Brute Force: Confusion Matrix: <pre>[[1 0 0 0 0 0] [2 0 0 0 0 0] [21 8 26 2 1 1] [2 0 0 0 0 0] [22 12 41 12 1 2] [3 4 9 1 0 0]]</pre>
Precision: 0.7590 Recall: 0.1479 F1-score: 0.1801 Overall, the LSTM model did not perform well in detecting DDoS. Possible Cause: Smaller Dataset.	Precision: 0.3813 Recall: 0.1637 F1-score: 0.1446 Overall, the LSTM model did not perform well in detecting Brute Force attack. Possible Cause: Smaller Dataset.

Figure 37. LSTM Results

Autoencoder Results:

DDoS: Confusion Matrix: <pre>[[115 0] [13 7]]</pre> Classification Report: <pre> precision recall f1-score support Normal 0.90 1.00 0.95 115 Anomalous 1.00 0.35 0.52 20 accuracy 0.90 135</pre>	Brute Force: Confusion Matrix: <pre>[[99 0] [55 9]]</pre> Classification Report: <pre> precision recall f1-score support Normal 0.64 1.00 0.78 99 Anomalous 1.00 0.14 0.25 64 accuracy 0.66 163</pre>
--	---

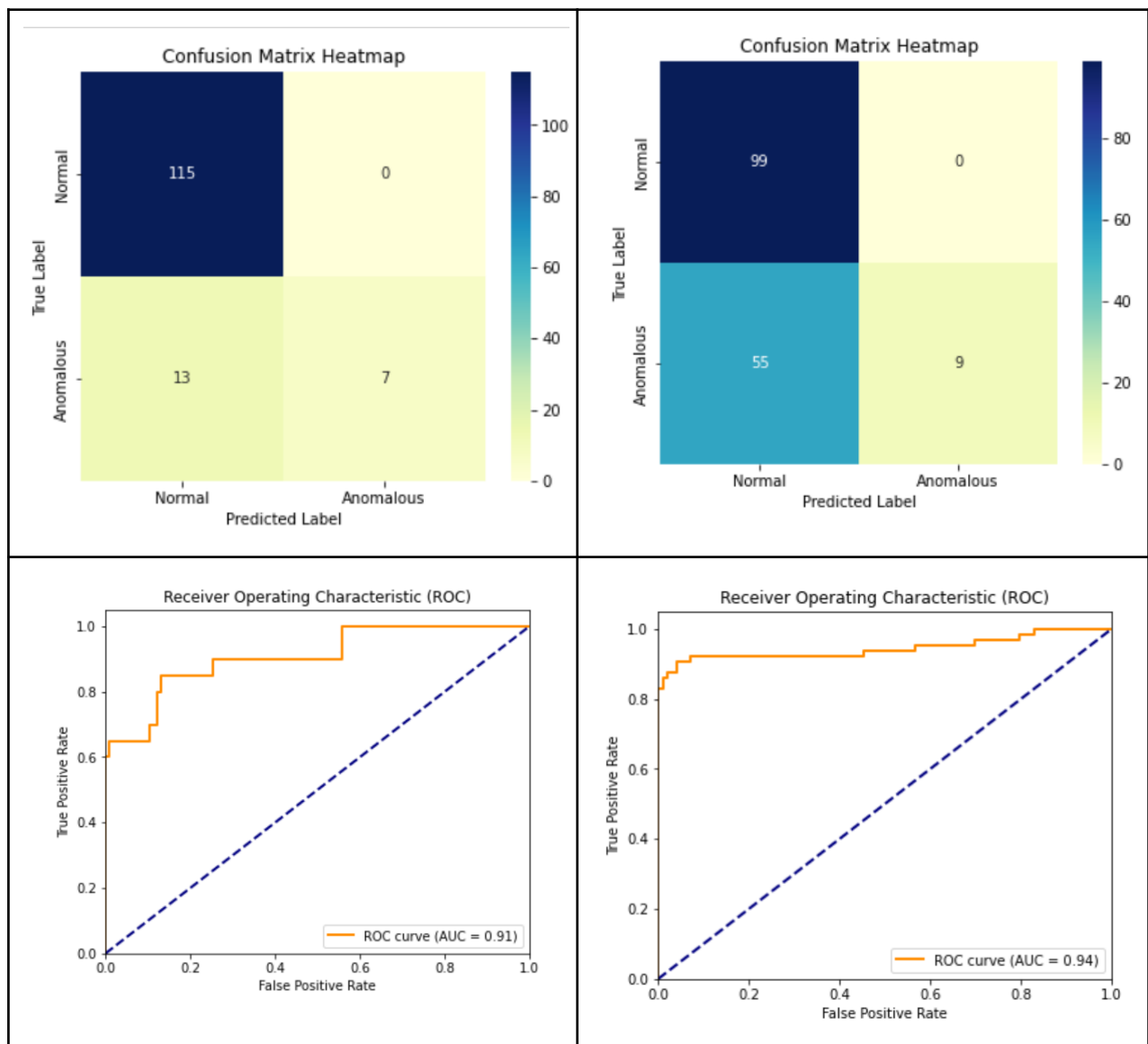


Figure 38. Autoencoder Results

Autoencoder Results with SMOTE:

DDoS:					Brute Force:				
Confusion Matrix:					Confusion Matrix:				
[[106 10]					[[82 18]				
[33 83]]					[39 62]]				
Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Normal	0.76	0.91	0.83	116	Normal	0.68	0.82	0.74	100
Anomalous	0.89	0.72	0.79	116	Anomalous	0.78	0.61	0.69	101
accuracy			0.81	232	accuracy			0.72	201

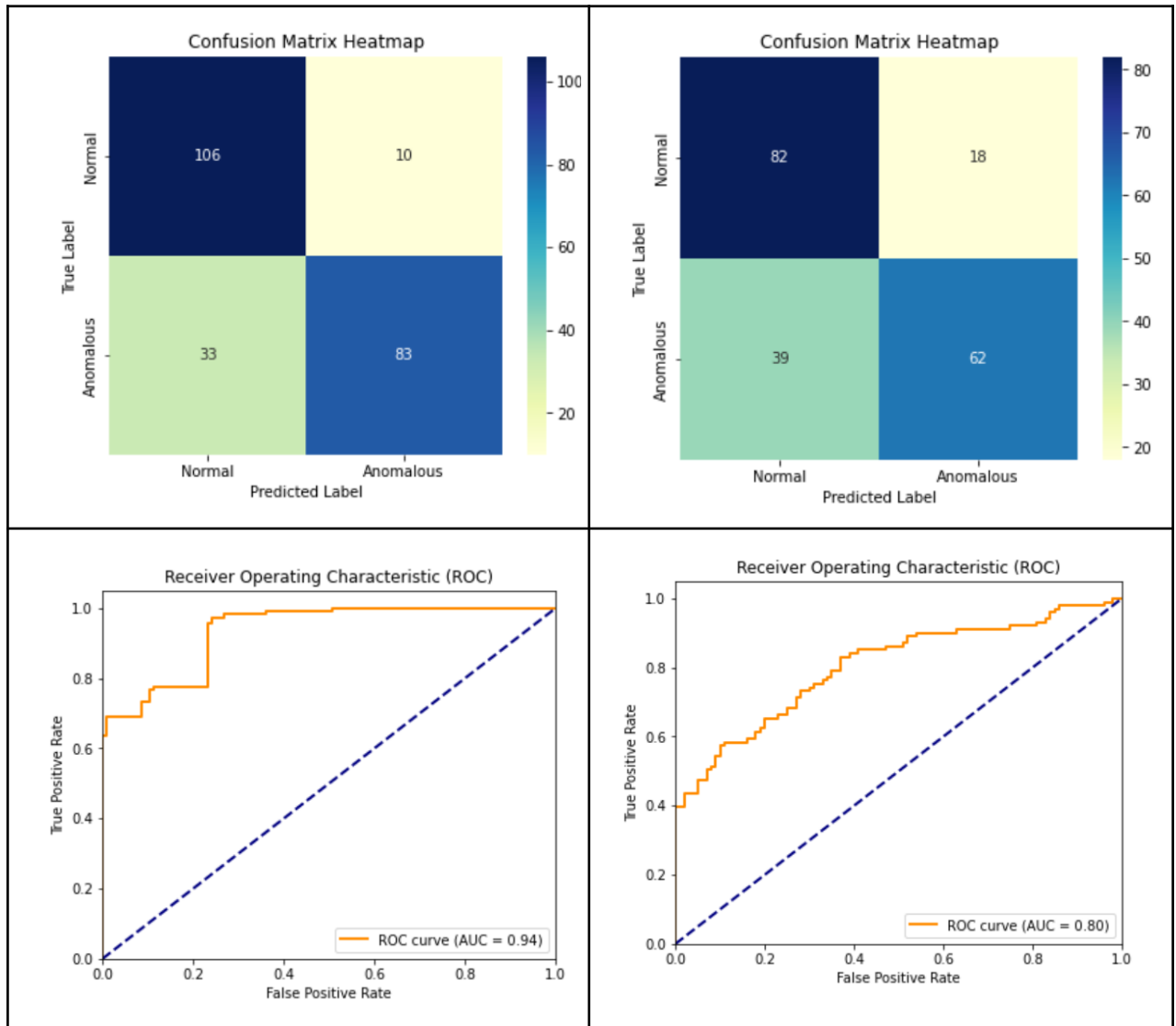


Figure 39. Autoencoder with SMOTE Results

LSTM results:

The LSTM accuracy in detecting DDoS attack is pretty good at a first glance but the poor recall makes it a bad model in this scenario where it is a priority to detect anomalous classes or attacks in simple terms. This resulted in a bad F1 score so. Even after tuning the hyperparameters like the threshold, it performed more or less the same in all the scenarios. In case of Brute Force attack the model wasn't a good option either. The possible reason might be the data we collected was too small to efficiently train the model for this kind of problem. Thus we had to select a different model to tackle this issue which was the Autoencoder.

Autoencoder results:

DDoS:

Before SMOTE:

The macro averages of precision (0.95), recall (0.68), and F1-score (0.73) show an imbalance in the performance across the two classes.

After SMOTE:

The macro averages of precision (0.83), recall (0.81), and F1-score (0.81) show a more balanced performance across the two classes after applying SMOTE.

Brute Force:

Before SMOTE:

The macro averages of precision (0.82), recall (0.57), and F1-score (0.51) show an imbalance in the performance across the two classes. The model is performing well for the normal instances but struggles to identify anomalous instances.

After SMOTE:

The macro averages of precision (0.73), recall (0.72), and F1-score (0.71) show a better balance in the performance across the two classes after applying SMOTE. The model has improved its performance in identifying anomalous instances and has a better balance between precision and recall for both classes.

Thus, the Autoencoder deep learning model turned out to be a better choice for our problem.

Chapter 5 - Conclusion and Future Work

This thesis provided a method for leveraging AWS to monitor and analyze cybersecurity attacks on microservice applications. The "sock-shop" application was set up on AWS EKS, customized for simulated cyberattacks, and put through a number of attack scenarios. Using Fluent Bit and AWS CloudWatch, simulation-related logs were gathered, preprocessed, and utilized to train and assess deep learning models for attack detection and categorization.

Dataset that was collected wasn't enough to train an LSTM Model to be effective to detect attacks. Autoencoder Model did a better job but still not good enough to detect an anomaly due to dataset limitations. After sampling the minority class the model was balanced and performed good enough to classify anomalies effectively.

The outcomes show that the suggested Autoencoder model is relatively accurate, precise, recallable, and F1-score efficient in identifying and classifying cybersecurity assaults. Future work will focus on collecting attack and baseline logs over an extensive period, increasing attack scenarios, optimizing the autoencoder architecture and training parameters, examining additional deep learning architectures along with with other anomaly detection algorithms, such as Isolation Forest, Local Outlier Factor, or One-Class SVM, adding real-time monitoring and alerting using a sliding window approach, allowing the model to adapt to new data and detect anomalies as they occur, and integrating with current security tools and platforms, and researching different cloud settings.

In summary, this thesis advances cybersecurity for microservice applications by demonstrating potential in strengthening cyber threat identification and analysis, ultimately assisting in defending these crucial systems from developing attacks.

REFERENCES

- [1] Turki Alharkan; Patrick Martin, "IDSaaS: Intrusion Detection System as a Service in Public Clouds" <https://ieeexplore-ieee-org.libproxy.csun.edu/document/6217486/>
- [2] Mohamed Almorsy; John Grundy; Amani S. Ibrahim, "CloudSec: A security monitoring appliance for Virtual Machines in the IaaS cloud model" <https://ieeexplore-ieee-org.libproxy.csun.edu/document/6008731/>
- [3] Anna L. Buczak; Erhan Guven "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection" <https://ieeexplore-ieee-org.libproxy.csun.edu/document/7307098/>
- [4] Matthias Haselmann; Dieter P. Gruber; Paul Tabatabai, "Anomaly Detection Using Deep Learning Based Image Completion" <https://ieeexplore-ieee-org.libproxy.csun.edu/document/8614226/>
- [5] Weaveworks. What are Microservices? <https://www.weave.works/>
- [6] Doug Chamberlain. Containers vs. Virtual Machines (VMs): What's the Difference? <https://blog.netapp.com/blogs/containers-vs-vms/>
- [7] Pavel Laskov, Patrick Dussel, Christin Schöfer, and Konrad Rieck. Learning intrusion detection: Supervised or unsupervised? In ICIAP, 2005.
- [8] Florian Schmidt, Florian Suri-Payer, Anton Gulenko, Marcel Wallschläger, Alexander W. Acker, and Odej Kao. Unsupervised anomaly event detection for cloud monitoring using online arima. 11 2018.
- [9] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. ACM Comput. Surv., 41(3):15:1–15:58, July 2009.
- [10] Kubernetes. Learn Kubernetes basics. <https://kubernetes.io/docs/tutorials/kubernetesbasics/>
- [11] AWS. Microservices on AWS. <https://docs.aws.amazon.com/whitepapers/latest/microserviceson-aws/introduction.html>